

About rulesets

Rulesets help you to control how people can interact with branches and tags in a repository.

Who can use this feature?

🔑 Anyone with read access to a repository can view the repository's rulesets. People with admin access to a repository, or a custom role with the "edit repository rules" permission, can create, edit, and delete rulesets for a repository.

📁 Rulesets are available in public repositories with GitHub Free and GitHub Free for organizations, and in public and private repositories with GitHub Pro, GitHub Team, and GitHub Enterprise Cloud. For more information, see [GitHub's plans](#).

Push rulesets are available for the GitHub Team plan in internal and private repositories, and forks of repositories that have push rulesets enabled.

 Copy as Markdown ▼

In this article

About rulesets

About rulesets and protected branches

Using ruleset enforcement statuses

About rule layering

About rulesets

A ruleset is a named list of rules that applies to a repository or to multiple repositories in an organization for customers on GitHub Team and GitHub Enterprise plans. You can have up to 75 rulesets per repository, and 75 organization-wide rulesets.

When you create a ruleset, you can allow certain users to bypass the rules in the ruleset. This can be users with a certain role, such as repository administrator, or it can be specific teams or GitHub Apps. For more information about granting bypass permissions, see [Creating rulesets for a repository](#).

For organizations on the GitHub Enterprise plan, you can set up rulesets at the organization level to target multiple repositories in your organization. See [Managing rulesets for repositories in your organization](#) in the GitHub Enterprise Cloud documentation.

You can use rulesets to target branches or tags in a repository or to block pushes to a repository and the repository's entire fork network.

Branch and tag rulesets

You can create rulesets to control how people can interact with selected branches and tags in a repository. You can control things like who can push commits to a certain branch, or who can delete or rename a tag. For example, you could set up a ruleset for your repository's `feature` branch that requires signed commits and blocks force pushes for all users except repository administrators.

For each ruleset you create, you specify which branches or tags in your repository the ruleset applies to. You can use `fnmatch` syntax to define a pattern to target specific branches and tags. For example, you could use the pattern `releases/**/*` to target all branches in your repository whose name starts with the string `releases/`. For more information on `fnmatch` syntax, see [Creating rulesets for a repository](#).

Push rulesets

With push rulesets, you can block pushes to a private or internal repository and that repository's entire fork network based on file extensions, file path lengths, file and folder paths, and file sizes.

Push rules do not require any branch targeting because they apply to every push to the repository.

Push rulesets allow you to:

- **Restrict file paths:** Prevent commits that include changes in specified file paths from being pushed.

You can use `fnmatch` syntax for this. For example, a restriction targeting `test/demo/**/*` prevents any pushes to files or folders in the `test/demo/` directory. A restriction targeting

`test/docs/pushrules.md` prevents pushes specifically to the `pushrules.md` file in the `test/docs/` directory. For more information, see [Creating rulesets for a repository](#).

- **Restrict file path length:** Prevent commits that include file paths that exceed a specified character limit from being pushed.
- **Restrict file extensions:** Prevent commits that include files with specified file extensions from being pushed.
- **Restrict file size:** Prevent commits that exceed a specified file size limit from being pushed.

About push rulesets for forked repositories

Push rules apply to the entire fork network for a repository, ensuring every entry point to the repository is protected. For example, if you fork a repository that has push rulesets enabled, the same push rulesets will also apply to your forked repository.

For a forked repository, the only people who have bypass permissions for a push rule are the people who have bypass permissions in the root repository.

About rulesets and protected branches

Rulesets work alongside any branch protection rules in a repository. Many of the rules you can define in rulesets are similar to protection rules, and you can start using rulesets without overriding any of your existing protection rules.

Rulesets have the following advantages over branch protection rules.

- Unlike protection rules, multiple rulesets can apply at the same time, so you can be confident that every rule targeting a branch in your repository will be evaluated when someone interacts with that branch. See [About rule layering](#).
- Rulesets have statuses, so you can easily manage which rulesets are active in a repository without needing to delete rulesets.
- Anyone with read access to a repository can view the active rulesets for the repository. This means a developer can understand why they have hit a rule, or an auditor can check the security constraints for the repository, without requiring admin access to the repository.
- You can create additional rules to control the metadata of commits entering a repository, such as the commit message and the author's email address. See [Available rules for rulesets](#) in the GitHub Enterprise Cloud documentation.

Using ruleset enforcement statuses

While creating or editing your ruleset, you can use enforcement statuses to configure how your ruleset will be enforced.

You can select any of the following enforcement statuses for your ruleset.

-  **Active:** your ruleset will be enforced upon creation.
-  **Disabled:** your ruleset will not be enforced.

About rule layering

A ruleset does not have a priority. Instead, if multiple rulesets target the same branch or tag in a repository, the rules in each of these rulesets are aggregated. If the same rule is defined in different ways across the aggregated rulesets, the most restrictive version of the rule applies. As well as layering with each other, rulesets also layer with protection rules targeting the same branch or tag.

For example, consider the following situation for the `my-feature` branch of the `octo-org/octo-repo` repository.

- An administrator of the repository has set up a ruleset targeting the `my-feature` branch. This ruleset requires signed commits, and three reviews on pull requests before they can be merged.
- An existing branch protection rule for the `my-feature` branch requires a linear commit history, and two reviews on pull requests before they can be merged.

The rules from each source are aggregated, and all rules apply. Where multiple different versions of the same rule exist, the result is that the most restrictive version of the rule applies. Therefore, the `my-feature` branch requires signed commits and a linear commit history, and pull requests targeting the branch will require three reviews before they can be merged.

Legal